

Render Farm

A self-running distributed job-processing demo built on real Redis and real BullMQ, depicting a render farm where frames flow through a shared queue and are processed by worker nodes running as separate OS processes.

Node 20+ / TypeScript

Express 5

BullMQ + Redis

Vite + React SPA

WebSocket transport

pnpm workspaces

Contents

01	System Overview	purpose, the four behaviors, design ethos
02	Architecture & Topology	control plane, data plane, view, contract
03	Process & Deployment Model	what runs, how many, which connections
04	The Wire Contract	shared types, constants, message shapes
05	Two Channels of Truth	QueueEvents vs telemetry, world-state merge
06	The Director Cycle Engine	pure reducer, effects, autonomous drama
07	Worker Node Lifecycle	spawn, process, heartbeat, exit
08	Crash Recovery Deep Dive	the headline mechanism and its timing
09	Autoscaling & Priorities	queue-depth thresholds, overtaking
10	Transport & Frontend	snapshot broadcast, four UI regions
11	Configuration Reference	every tunable and its role
12	Failure Modes & Testing	edge cases, test strategy, invariants

1 System Overview

The demo is a live, looping visualization of how a distributed job-processing system behaves under load, failure, and scaling pressure - using genuine infrastructure rather than a simulation of it.

Frames (the unit of work, standing in for a rendered image) enter a shared BullMQ queue and are pulled by worker nodes. Each worker is a real forked OS process with its own Redis connection and its own BullMQ `Worker`. The system runs on an accelerated clock: a cycle seeds a batch of frames into an empty board, drains every frame to completion, holds a brief "complete" beat, resets, and repeats forever. On a 5 to 10 second cadence an autonomous Director injects scripted drama - crashing a node, forking a new one, retiring an idle one - so an observer always has something to watch.

The work itself is simulated (timed delays in two stages), but everything that makes the behavior *distributed* is real: the queue is real, the inter-process boundary is real, a crash is a real `SIGKILL`, and recovery is real BullMQ stalled-job detection.

The four behaviors it demonstrates

Behavior	How it actually works	Where you see it on screen
Load balancing	A single shared BullMQ queue. Whichever worker is idle pulls the next frame; there is no assignment logic and no faking.	Frames spread themselves across the node strip and the Kanban board's in-flight columns.
Fault tolerance	A crash is a real <code>SIGKILL</code> of a worker child mid-frame. Its Redis lock expires, and BullMQ's stalled-job detection re-queues the orphaned frame to a surviving worker.	A node card goes red, its frame slides back to <code>QUEUED</code> , then completes elsewhere. The log narrates the crash and the re-queue.

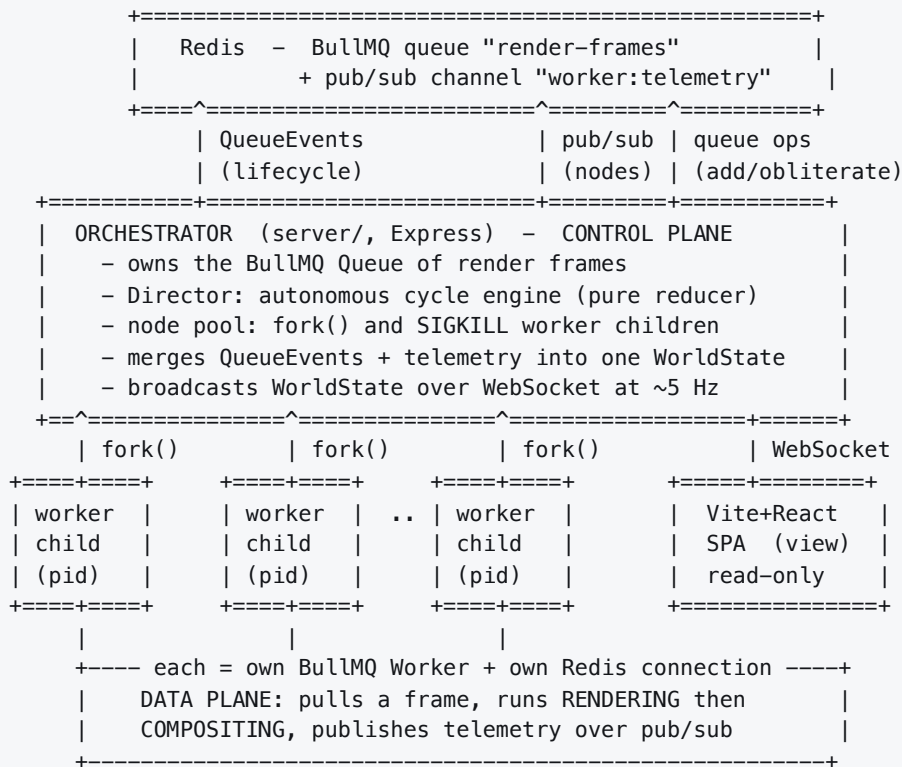
Behavior	How it actually works	Where you see it on screen
Autoscaling	The Director watches queue depth. A deep backlog forks a new worker process; a draining queue gracefully retires an idle one.	Node cards appear and disappear in the strip. The log records <i>autoscaling up</i> and <i>autoscaling down</i> .
Job priorities	A fraction of frames are enqueued with BullMQ <code>priority</code> and overtake normal frames still waiting in the queue.	Purple-badged <code>PRIORITY</code> cards jump toward the front of the <code>QUEUED</code> column.

Design ethos

- **Authenticity over illusion.** The project deliberately pays the cost of real processes and a real message broker so that the emergent behavior - load spreading, recovery latency, backlog-driven scaling - is a property of the system rather than a script. The only thing faked is the render work itself, which is timed sleep in two stages.
- **One command path for autonomous and manual actions.** The operator controls (inject, kill, pause, reset) feed the exact same effect-application code the Director uses on its own ticks, so there is a single tested path per action rather than a parallel manual implementation.
- **A pure decision core with an impure driver.** Every scheduling decision lives in a pure reducer that takes state plus an observed snapshot and returns the next state plus a list of effects. All side effects (forking, killing, enqueueing, logging) are applied outside that function, which keeps the entire cycle engine unit-testable without Redis.
- **Full-snapshot transport.** At demo scale (at most roughly twenty frames and six nodes) a complete world snapshot is small, so the server broadcasts the whole state at 5 Hz instead of computing diffs. Every message then doubles as late-join state for a browser that connects mid-cycle.

2 Architecture & Topology

Four workspaces map onto three planes plus a shared contract: a control plane that decides and aggregates, a data plane that does the work, a view that only renders, and a package of types all three speak.



The four workspaces

Workspace	Plane	Responsibility
server/	Control plane	Owns the queue, runs the Director, manages the worker pool, subscribes to both truth channels, aggregates <code>WorldState</code> , and serves the UI over WebSocket. This is the only process that forks workers and the only one that mutates the queue.
worker/	Data plane	Each node is an independently forked Node process running a BullMQ <code>Worker</code> at concurrency 1. It runs a frame through the render then composite stages and publishes fine-grained node status and progress over the telemetry channel. It never talks to the browser and never touches the WebSocket.
web/	View	A Vite and React single-page app. It never touches Redis and holds no business logic; it renders whatever <code>WorldState</code> snapshot it last received and sends operator commands back over the same socket.
shared/	Contract	The <code>@demo/shared</code> package: the queue name, stage names, telemetry channel, event levels, command types, and every wire type. All three runtime workspaces depend on it, so the contract cannot drift between them.

Dependency direction

Dependencies flow strictly one way. The control plane depends on `@demo/shared` and drives Redis and the worker pool; workers depend on `@demo/shared` and Redis only; the view depends on `@demo/shared` and the WebSocket only. No workspace reaches sideways into another's source - the shared contract is the single seam between them, which is what lets a worker be launched as a genuinely separate process without importing any orchestrator code.

3 Process & Deployment Model

At steady state the machine runs one orchestrator process, one Vite dev server, a Redis server, and between two and six worker child processes - each worker forked by the orchestrator, not by a supervisor or container runtime.

What runs, and how it is launched

- **Redis** on `localhost:6379`, started either natively (`npm run redis:start`, the path used here) or via `docker compose up -d (redis:7-alpine)`. It hosts both the BullMQ queue data structures and the pub/sub telemetry channel; there is no other datastore and no persistence beyond Redis.
- **The orchestrator** (`server/src/index.ts`) binds Express plus a `ws` WebSocket server on port `3001`, opens three separate Redis connections (one for the queue, one for `QueueEvents`, one for the telemetry subscription), spawns the initial `MIN_NODES` workers, and starts the Director's randomized tick loop.
- **Worker children** are created with Node's `fork()` pointing at `worker/src/index.ts`, launched under `tsx` via `execArgv: ['--import', 'tsx']`. The pool injects each child's identity and timing knobs through environment variables (`NODE_ID`, `REDIS_URL`, `STAGE_MS`, `LOCK_DURATION_MS`, `STALLED_INTERVAL_MS`, `MAX_STALLED_COUNT`, `HEARTBEAT_INTERVAL_MS`). Each child opens two Redis connections of its own: one for the BullMQ worker (`maxRetriesPerRequest: null`, as BullMQ requires) and one dedicated publisher for telemetry.
- **The web SPA** is served by Vite on port `5173` and connects outward to the orchestrator's WebSocket. Nothing in the browser tier is privileged; closing it has no effect on the running cycle.

CONNECTION BUDGET

The orchestrator holds three Redis connections; every worker holds two. A cycle running at the ceiling of six workers therefore has roughly $3 + (6 \times 2) = 15$ live Redis connections. This is deliberate: BullMQ's blocking commands and pub/sub each need their own connection, and sharing them across concerns would serialize unrelated traffic.

Node identity and the pool counter

The node pool assigns ids from a monotonic counter: `node-1`, `node-2`, and so on. The counter never resets within a process lifetime, so a retired or crashed `node-3` is never reused; the next spawn is always a fresh higher number. This makes the event log unambiguous - every id refers to exactly one process that ever existed - at the cost of ids climbing over a long-running demo.

Lifecycle boundaries

- **Graceful retirement** sends `SIGTERM`. The worker clears its heartbeat, closes its BullMQ worker cleanly, and exits 0. The pool's exit handler treats a clean exit as "not a crash" and removes the node from the world immediately.
- **Hard crash** sends `SIGKILL`. There is no cleanup; the process dies mid-frame with its Redis lock still held. The exit handler treats a signal kill (or any nonzero exit) as a crash, marks the node `crashed`, lets it linger on screen for `1500 ms` so the failure is visible, then removes it.
- **Orchestrator shutdown** (`SIGTERM` / `SIGINT`) stops the Director timer, `SIGKILL`s every worker child, and closes the queue, the queue-events listener, and all three Redis connections before exiting.

4 The Wire Contract

The `@demo/shared` package is the single source of truth for every value and shape that crosses a process boundary. Because all three runtimes import it, a change to the contract is a compile-time event everywhere at once.

Constants

Constant	Value	Meaning
<code>QUEUE_NAME</code>	<code>"render-frames"</code>	The one BullMQ queue every worker binds to.
<code>TELEMETRY_CHANNEL</code>	<code>"worker:telemetry"</code>	The Redis pub/sub channel workers publish to and the orchestrator subscribes to.
<code>STAGES</code>	<code>QUEUED, RENDERING, COMPOSITING, DONE</code>	The ordered Kanban columns; also a frame's <code>stage</code> field.
<code>EVENT_LEVELS</code>	<code>info, success, warn, danger</code>	Severity of an event-log line, which drives its color.
<code>COMMAND_TYPES</code>	<code>pause, resume, inject, killNode, reset</code>	The operator commands the browser may send.

Core types

Frame - the unit of work

```
interface Frame {
  id: string;      // "<cycle>--<seq>"
  cycle: number;
  stage: Stage;    // QUEUED..DONE
  nodeId: string | null;
  pct: number;     // 0..100
  priority: boolean;
  failed: boolean; // exhausted retries
}
```

TelemetryMsg - worker to orchestrator

```
interface TelemetryMsg {
  nodeId: string;
  pid: number;
  state: NodeState;
  frameId: string | null;
  stage: Stage | null;
  pct: number;
  priority: boolean;
  completed: number;
}
```

WorkerNode - a live process

```
interface WorkerNode {
  id: string;      // "node-N"
  pid: number;
  state: NodeState;
  frameId: string | null;
  pct: number;
  completed: number;
}
```

WorldState - the broadcast snapshot

```
interface WorldState {
  cycle: number;
  phase: 'seeding' | 'running'
    | 'complete' | 'paused';
  frames: Frame[];
  nodes: WorkerNode[];
  events: LogEvent[];
  totals: { done; total };
}
```

Node states form the visual vocabulary of the node strip: `idle`, `rendering`, `compositing`, `spawning`, and `crashed`. The first four come from telemetry the worker publishes about itself; `crashed` is applied by the orchestrator when a child exits unexpectedly, because a killed process cannot announce its own death.

FRAME IDENTITY IS QUEUE IDENTITY

A frame's `id` is used directly as the BullMQ `jobId`. That single decision is what lets a lifecycle event (which carries only a `jobId`) be folded straight into the matching frame with no side lookup table - the job and the frame are the same entity under two names.

5 Two Channels of Truth

The orchestrator reconciles two independent event streams into one coherent world: BullMQ's queue lifecycle for *where a frame is*, and a pub/sub telemetry channel for *what each node is doing*. Each answers a question the other cannot answer cheaply.

Why two channels rather than one

BullMQ's `QueueEvents` is authoritative for job lifecycle: a frame was added, completed, stalled, or failed. It is reliable and it is the same signal that drives real recovery. But it does not cheaply tell you which node currently owns a frame, what stage that frame is in, or how far along it is. That fine-grained, high-frequency detail rides a dedicated Redis pub/sub channel instead, where a lost message simply means one slightly stale progress bar rather than a lost state transition.

Channel	Carries	Folded by
QueueEvents authoritative, low-frequency	<code>added</code> creates a QUEUED frame and bumps the total; <code>completed</code> and <code>failed</code> both move it to DONE (failed keeps a flag); <code>stalled</code> re-queues it back to QUEUED with no owner.	<code>applyQueueEvent</code> - moves a frame between stages and updates <code>totals</code> .
worker:telemetry best-effort, high-frequency	Per-node state, the frame it owns, current stage, live percent, per-node completed count, and idle heartbeats.	<code>applyTelemetry</code> - upserts the node and, if it owns a frame, syncs that frame's stage, owner, and percent.

The merge and the throttle

Both folds write into a single in-memory `WorldStore`. The store is a plain reducer over immutable snapshots: every handler takes the current `WorldState` and returns a new one, so there is no shared mutable state to guard. A separate broadcaster reads the latest snapshot on a fixed timer and pushes it to every open socket at `BROADCAST_HZ` (5 Hz by default), decoupling the write rate of the two noisy input channels from the send rate to browsers. A newly connected browser is sent a full snapshot immediately on connect, so opening the page mid-cycle shows correct state at once.



ORDERING & RECONCILIATION

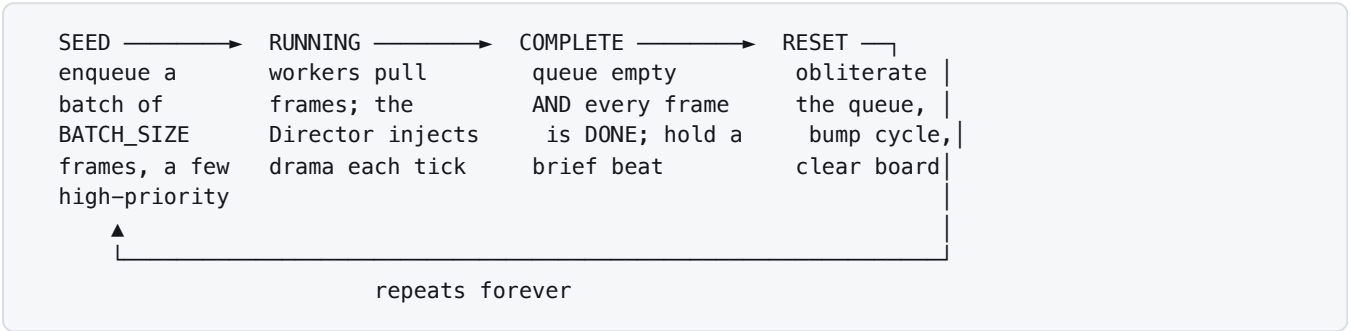
The two channels are not ordered relative to each other. Telemetry may report a node "rendering frame f2-7" a beat before or after the `active` lifecycle transition. The merge is written to tolerate this: `applyTelemetry` only syncs a frame it can find and that reports a stage, and a `stalled` event unconditionally resets ownership - so a late telemetry straggler for an orphaned frame cannot pin it to a dead node.

6 The Director Cycle Engine

The Director is the autonomous brain. It is split cleanly into a pure reducer that decides what should happen and an impure driver that makes it happen - the same split that makes the whole engine testable without touching Redis or forking a process.

The cycle as a state machine

A cycle is one batch of frames from an empty board to all-complete. The Director walks a small phase machine, advanced by a timer that fires at a random interval between `TICK_MIN_MS` and `TICK_MAX_MS` :



The pure reducer

Every decision is made by `reduceDirector(state, action, ctx)` , which returns the next state plus a list of effects and performs no side effects itself. Its inputs are the current phase, the incoming action (a tick, or a manual command), and an observed context snapshot - queue depth, active count, node count, the ids of busy nodes, and two random rolls injected from outside so the function stays deterministic under test.

In phase / action	The reducer decides
seeding	Emit a single <code>seed</code> effect for <code>BATCH_SIZE</code> frames, then advance to <code>running</code> .
<code>running</code> , work remains	Consider one scaling effect (spawn if backlog exceeds the up-threshold and below max nodes; kill if the queue drains below the down-threshold and above min nodes) and, independently, a probabilistic crash if more than one node is busy.
<code>running</code> , nothing remains	Advance to <code>complete</code> .
complete	Emit <code>resetQueue</code> , increment the cycle number, return to <code>seeding</code> .
pause / resume	Flip the paused flag with no effects. While paused, a tick returns state unchanged and emits nothing.

The impure driver

`runDirector` owns everything the reducer refuses to touch. It builds the observed context by querying live BullMQ job counts and the current node list, calls the reducer, then applies each returned effect in order: `seed` enqueues frames (assigning each a `jobId` , a high or normal priority, and generous retry attempts), `spawn` forks a worker, `kill` retires an idle one, `crash SIGKILL` s a busy one, and `resetQueue` obliterates the queue and clears the board. It also owns the frame-id sequence and a small map remembering which ids were seeded high-priority, so the `added` lifecycle handler can recover a frame's priority.

RANDOMNESS LIVES AT THE EDGE

The crash decision reads two random values (`crashRoll` , `targetRoll`) from the context, not from inside the reducer. A crash fires only when `crashRoll` falls under the per-tick probability of `0.25` and more than one node is busy; the target is then chosen from the busy set. Injecting the rolls keeps the reducer pure and lets tests pin an exact crash or an exact no-crash by supplying the rolls directly.

Manual commands share the path

Operator commands enter through the WebSocket and are routed to the same driver: pause, resume, and reset flow through `dispatch` into the reducer; inject calls the same `seed` the Director uses; kill calls a dedicated `killNodeNow` . Crucially, a manual kill works even while paused - pause freezes the Director's own drama, not the operator's hand - and it prefers a busy node but falls back to any node so the button is never a no-op.

7 Worker Node Lifecycle

A worker is a small, single-purpose process: bind a BullMQ worker at concurrency one, run each frame through two timed stages while narrating progress, and announce itself when idle. It knows nothing about cycles, scaling, or the browser.

The stage pipeline

Each frame runs through two stages in strict order, `RENDERING` then `COMPOSITING` , each simulated as roughly `STAGE_MS` of work. During a stage the worker ramps a progress percent and publishes a telemetry snapshot as it climbs, so the browser's per-node progress bar and the frame's own percent advance smoothly. When both stages finish, BullMQ marks the job `completed` , the worker increments its per-node completed count, and it publishes an idle snapshot to signal it is ready for the next frame.

```
pull frame → (concurrency = 1) → RENDERING → emit {stage,pct, nodeId} as pct ramps 0..100 → COMPOSITING → emit {stage,pct, nodeId} as pct ramps 0..100 → completed completed++, publish idle
```

Heartbeat and idle presence

An idle worker re-announces itself every `HEARTBEAT_INTERVAL_MS` on the telemetry channel. This matters because telemetry is best-effort: without a heartbeat, a worker that finished its last frame and is waiting for the next would silently vanish from the strip. While it is actively processing, the progress ticks already prove it is alive, so the heartbeat only fires when idle.

Exit paths and how the orchestrator reads them

Signal	Worker behavior	Orchestrator's exit handler
<code>SIGTERM</code> graceful retire	Clears the heartbeat timer, closes the BullMQ worker (finishing or cleanly releasing its frame), exits 0.	Sees a clean exit, classifies it as "not a crash", and removes the node from the world immediately with no red state.

Signal	Worker behavior	Orchestrator's exit handler
<code>SIGKILL</code> hard crash	Dies instantly. No cleanup. Its held frame lock is left to expire.	Classifies a signal kill (or any nonzero code) as a crash, marks the node <code>crashed</code> , logs it, lets the red card linger <code>1500 ms</code> , then removes it.

A CRASH IS GENUINELY A CRASH

The `Kill a node` control and the Director's random crash both send a real `SIGKILL` to a real OS process. There is no soft-crash shortcut in this codebase - the design spec kept one as a documented fallback, but real-kill recovery proved reliable on the accelerated clock and the fallback was never built. What you watch recover is authentic BullMQ stalled-job detection.

8 Crash Recovery Deep Dive

This is the headline mechanism and the one piece of timing tuned specifically for a watchable demo. A killed worker leaves an orphaned frame; BullMQ notices the abandoned lock and hands the frame to a survivor - and the whole round trip is engineered to complete inside a few seconds.

The sequence, step by step

t	Event	What is happening underneath
0.0s	Worker is mid-frame, holding the job's Redis lock, renewing it as it works.	BullMQ maintains a lock with TTL <code>LOCK_DURATION_MS</code> , renewed while the processor runs.
0.0s	<code>SIGKILL</code> arrives; the process dies instantly.	No cleanup runs. The lock is now held by a process that no longer exists and will not be renewed again.
~0s	Orchestrator's exit handler marks the node <code>crashed</code> and logs it.	Driven by the child <code>exit</code> event, independent of Redis. The frame's own recovery has not started yet.
~4s	The abandoned lock expires.	After <code>LOCK_DURATION_MS</code> (4000) with no renewal, the job is eligible to be seen as stalled.
~4–6s	A surviving worker's stalled check finds the orphan and re-queues it.	Each worker runs a stalled sweep every <code>STALLED_INTERVAL_MS</code> (2000). The next sweep after lock expiry emits <code>stalled</code> ; the frame slides back to QUEUED.
~6s+	An idle worker pulls the re-queued frame and completes it.	Normal load balancing resumes. The frame finishes on a different node; the log shows the re-queue then the completion.

THE RECOVERY-WINDOW FORMULA

Worst-case recovery latency is bounded by $\text{LOCK_DURATION_MS} + \text{STALLED_INTERVAL_MS} = 4000 + 2000 = 6000$ ms, landing recovery inside a roughly 4 to 8 second window. BullMQ's stock defaults (a 30-second stalled interval) would push this past half a minute - far too slow to watch - which is exactly why these two knobs are tightened for the demo and validated by an integration test.

Retry headroom, so a crash never fails a frame

Each frame is enqueued with `JOB_ATTEMPTS` (20) attempts and workers run with `MAX_STALLED_COUNT` (10). These are set deliberately high so that a frame crashed and re-queued several times still has plenty of

headroom to succeed rather than being marked permanently failed. The generous budget is what makes a cascade of crashes recoverable instead of fatal.

The deadlock-proof escape hatch

A frame that somehow exhausts all its attempts (reachable only with the retry budget forced down, for example `MAX_STALLED_COUNT=0 JOB_ATTEMPTS=1`) does not vanish and does not wedge the cycle. BullMQ emits `failed`; the orchestrator still moves the frame into `DONE` but preserves a `failed` flag, which the board renders as a red-bordered `FAILED` card and the log records as *failed permanently*. Because a failed frame still counts as done, the "all frames complete" condition can always be reached and the cycle can never deadlock waiting on a frame that will never finish.

9 Autoscaling & Priorities

Autoscaling on queue depth

The Director scales the worker pool reactively from a single observed number - the queue depth, defined as waiting plus prioritized jobs - against two strict thresholds, clamped to a floor and ceiling.

- **Scale up** forks one new worker when the queue depth strictly exceeds `SCALE_UP_DEPTH` (6) and the pool is below `MAX_NODES` (6). A deep backlog is thus met by more parallelism, one node per qualifying tick.
- **Scale down** gracefully retires one idle worker when the queue depth drops strictly *below* `SCALE_DOWN_DEPTH` (2) and the pool is above `MIN_NODES` (2). The pool only ever kills a node reporting `idle`, so scaling down never interrupts in-flight work.
- **The comparisons are strict on purpose.** "Exceeds" and "drains below" are exact boundary semantics pinned by unit tests, so a depth sitting exactly on a threshold does not oscillate the pool.
- **At most one scaling action per tick.** Up and down are mutually exclusive branches, so the pool moves by one node at a time and cannot thrash up and down within a single decision.

```
queueDepth = waiting + prioritized
```

```
depth > 6  and nodes < 6  → spawn one worker      (autoscaling up)
depth < 2  and nodes > 2  → retire one idle node  (autoscaling down)
otherwise                                     → hold steady
```

Priorities and overtaking

A configurable fraction of seeded frames are flagged high-priority and enqueued so they jump ahead of normal frames still waiting - a real property of the shared queue, not a display trick.

- **Seeding assigns priority probabilistically.** Each seeded frame is high-priority with probability `HIGH_PRIORITY_RATIO` (0.15), so a typical batch of sixteen carries two or three priority frames.
- **Lower BullMQ number means higher priority.** High-priority frames are enqueued with priority `1`, normal frames with priority `5`. When a worker frees up it pulls the lowest-numbered waiting job, so a priority frame overtakes normal frames that were queued before it.
- **Priority is remembered for the lifecycle handler.** The `added` event carries only a job id, so the driver keeps a small id-to-priority map from seed time; the handler reads it to render the incoming frame with the correct badge.

- **Overtaking is visible, not instantaneous.** A priority frame cannot preempt a frame already rendering (workers finish the frame they hold); it jumps the *waiting* line, so on screen it slides toward the front of QUEUED and enters processing ahead of its neighbors.

10 Transport & Frontend

Transport: full snapshots over one WebSocket

A single bidirectional WebSocket carries world snapshots down to the browser and operator commands back up. The server never sends diffs - it sends the whole world, framed so every message is also valid late-join state.

- **Downstream:** the broadcaster sends a full `{ type: 'snapshot', state }` immediately to each socket on connect, then re-sends the latest snapshot to all clients on a fixed timer at `BROADCAST_HZ` (5 Hz). A browser opening the page mid-cycle is correct on its first message.
- **Upstream:** the browser sends JSON commands (pause, resume, inject, killNode, reset) on the same socket. Malformed input is caught and ignored rather than allowed to crash the orchestrator, and unknown command types are dropped by the router.
- **Resilience:** the client reconnects on close with exponential backoff from `500 ms` up to a `8000 ms` ceiling, resetting the delay once a connection reopens, so a server restart heals without a page reload.
- **Why snapshots win at this scale:** with at most roughly twenty frames and six nodes a snapshot is a few kilobytes; diffing would add reconciliation complexity and a late-join code path for no bandwidth saving worth having.

The four UI regions

The SPA is one page driven entirely by the last snapshot; it holds no derived business state of its own.

Region	What it renders from the snapshot
Kanban board the primary graphic	One card per frame, placed in its <code>stage</code> column. Cards slide between columns on stage transitions, carry their owning node while in flight, wear a badge when high-priority, and turn red when failed. On a crash, orphaned cards slide back to QUEUED.
Node strip	One card per live process showing pid, state, current frame, a live progress bar, and a completed count. Cards appear and disappear as the pool scales; a crashed node flashes red before it is removed.
Event log	A scrolling, color-coded feed - the human-readable narration of what the graphics are doing, with severity mapped to color (info, success, warn, danger).
Control bar	The operator buttons (pause / resume, inject five, kill a node, reset), each sending a command on the shared socket.

ACCESSIBILITY BUILT IN

Motion respects `prefers-reduced-motion` - slide animations become instant state changes rather than merely shortened ones. The UI uses semantic HTML and native `<button>` controls, and targets WCAG AA contrast throughout, so the visualization is legible without relying on motion or color alone.

11 Configuration Reference

Every timing and threshold knob lives in `server/src/config/tunables.ts` and is overridable by an environment variable of the same upper-snake name. The defaults are chosen for an accelerated, watchable clock rather than production realism.

Env var	Default	Role in the system
<code>BATCH_SIZE</code>	<code>16</code>	Frames seeded at the start of every cycle.
<code>HIGH_PRIORITY_RATIO</code>	<code>0.15</code>	Probability each seeded frame is flagged high-priority.
<code>TICK_MIN_MS</code> / <code>TICK_MAX_MS</code>	<code>5000</code> / <code>10000</code>	Bounds of the randomized interval between Director drama ticks.
<code>STAGE_MS</code>	<code>2500</code>	Simulated work per stage; a frame spans two stages, so roughly 5s end to end.
<code>MIN_NODES</code> / <code>MAX_NODES</code>	<code>2</code> / <code>6</code>	Autoscale floor and ceiling; also the count spawned at startup (min).
<code>SCALE_UP_DEPTH</code>	<code>6</code>	Queue depth that must be strictly exceeded to fork a worker.
<code>SCALE_DOWN_DEPTH</code>	<code>2</code>	Queue depth the pool must strictly drain below to retire an idle worker.
<code>LOCK_DURATION_MS</code>	<code>4000</code>	Worker lock TTL; a crashed worker's frame becomes recoverable after this expires.
<code>STALLED_INTERVAL_MS</code>	<code>2000</code>	How often each worker sweeps for stalled (orphaned) jobs.
<code>HEARTBEAT_INTERVAL_MS</code>	<code>2000</code>	How often an idle worker re-announces itself on the telemetry channel.
<code>JOB_ATTEMPTS</code>	<code>20</code>	Retry attempts per frame; high so re-queued frames succeed rather than fail.
<code>MAX_STALLED_COUNT</code>	<code>10</code>	How many stalls a job tolerates before BullMQ fails it permanently.
<code>BROADCAST_HZ</code>	<code>5</code>	WorldState snapshot broadcast rate to all connected browsers.
<code>PORT</code>	<code>3001</code>	Orchestrator HTTP and WebSocket port.
<code>REDIS_URL</code>	<code>redis://127.0.0.1:6379</code>	Redis endpoint shared by orchestrator and every worker.

THE ONE COUPLED PAIR

`LOCK_DURATION_MS` and `STALLED_INTERVAL_MS` are the only knobs that must be tuned together: their sum sets the crash-recovery window. Raising either widens the gap between a kill and its visible recovery; lowering them too far risks a worker's own lock being declared stalled mid-frame. The shipped pair (4000 + 2000) is the validated sweet spot for the accelerated clock.

Ports at a glance

Service	Port
Web SPA (Vite)	5173
Orchestrator HTTP + WebSocket	3001
Redis	6379

12 Failure Modes & Testing

Invariants the system holds

- **A cycle always terminates.** Because every terminal outcome - completed or permanently failed - lands a frame in DONE and counts toward the total, the "all frames done" condition is always reachable and no cycle can wedge on a frame that will never finish.
- **The queue has a single owner.** Only the orchestrator adds, obliterates, or inspects the queue; workers only pull. This keeps queue mutation in one place and out of the racing data plane.
- **Node ids are unique for all time.** The monotonic pool counter guarantees a crashed or retired id is never reused, so every log line names exactly one process.
- **The view cannot corrupt state.** The browser is strictly downstream of the world store and upstream only through validated commands; malformed input is dropped, not applied.

Known edge cases and how they are handled

Situation	Handling
A worker killed while idle (scale-down race).	The pool only targets nodes reporting <code>idle</code> for graceful retire; a busy node is never chosen for scale-down.
A crash while only one node is busy.	The Director's crash guard requires more than one busy node, so it never orphans work that no survivor could pick up.
Kill pressed while paused.	Deliberately still fires - pause freezes autonomous drama, not the operator - and falls back to any node if none are busy.
A late telemetry message for a re-queued frame.	The <code>stalled</code> fold resets frame ownership, and telemetry only syncs a frame that still reports a stage, so a straggler cannot re-pin an orphan to a dead node.
Browser connects mid-cycle.	Served a full snapshot on connect, so first paint is correct.

Test strategy

The engine's decisions are pure functions, so they are covered without Redis, and the genuinely distributed behavior is covered against real Redis and real BullMQ.

- **Unit (no Redis):** the Director reducer - phase transitions, pause-gated crash selection, and the exact strict autoscale boundaries - plus the world-state merge folds, the worker's two-stage ordering and full progress ramp, and the WebSocket command router.
- **Integration (real Redis, run sequentially because the files share one queue):** the headline `SIGKILL` case asserts a frame stalls, re-queues, and is completed by the *surviving* worker; plus plain lifecycle (added to active to completed), priority overtaking, load balancing across two workers, the idle heartbeat, and graceful `SIGTERM` shutdown.

- **Web (jsdom):** column-stage mapping, priority ordering and badge, node tags and strip fields, event-log severity coloring, and control-bar command dispatch.

TEST / DEMO SHARE ONE REDIS

The running demo and the server integration tests use the same Redis queue, and the tests flush Redis. Stop `npm run dev` before running server integration tests, or a live demo will steal test jobs and the tests will wipe the demo's state out from under it.